BDA Week 7 — Recommendation Systems

Bridge module · pre-mid topic, post-mid exam material · numerical-trace heavy

Bridge module · pre-mid topic, post-mid exam material · numerical-trace heavy

Topics covered in this PDF:

- Amazon
- Netflix
- Google News

CS-404 Big Data Analytics · Dr. Syed Imran Ali · Final ~ 2026-05-16

Prepared as a comprehensive professor-grade study tutor.

Contents

1. §1 — Beginning Key Notes (Study Compass)
2. §2 — What Is a Recommender System?
3. §3 — The Utility Matrix
4. §4 — Content-Based Filtering
5. §5 — Collaborative Filtering (CF)
6. §6 — Implementing Cosine Similarity Properly
7. §7 — Baseline Predictors
8. §8 — Hybrid Approaches
9. §9 — Evaluation
10. §10 — Six Worked Numerical Examples
11. §11 — Practice Questions (15)
12. §12 — Full Worked Answers
13. §13 — Ending Key Notes (Revision Cards)
14. §14 — Formula & Algorithm Reference

Why this PDF matters.

Recommendation Systems was technically introduced before the midterm, but the examiner's solution pattern from the midterm proves he loves multi-step numerical traces — and recommender systems are PERFECT for that. Pearson similarity, cosine similarity, item-item rating prediction, and RMSE are textbook 15–25 mark numerical questions. Master §5–§10 here and you have another 20–25 marks locked in for the final on top of PageRank.

§1 — Beginning Key Notes (Study Compass)

These are the ten load-bearing ideas you must walk into the exam owning. Every numerical question in this module reduces to applying one of them.

1. **Two columns, one user-item matrix.** A recommender system's input is a sparse **utility matrix** U where $U[x][i]$ = user x's rating of item i. Most cells are blank. The job is to fill in the blanks intelligently.
2. **Three problems, in order.** (i) *Gather* ratings (explicit asks vs implicit signals). (ii) *Extrapolate* unknown ratings. (iii) *Evaluate* predictions (usually with RMSE).
3. **Two big algorithm families. Content-based** uses item features. **Collaborative filtering** uses the rating matrix alone. Latent-factor (matrix factorization) is a third family — out of scope for this module.
4. **Content-based recipe.** Build an *item profile* (vector of features, e.g. TF-IDF of words). Build a *user profile* by averaging the profiles of items the user liked. Predict $u(x, i) = cos(userProfile_x, itemProfile_i)$.
5. **Collaborative filtering — User-User.** $sim(x, y)$ via Pearson correlation on co-rated items. Predict $\hat{} r(x, i) = \Sigma_{y \in N} sim(x, y) \cdot r(y, i) / \Sigma_{y \in N} |sim(x, y)|$ over the k most similar users N who rated i.
6. **Collaborative filtering — Item-Item.** Same machinery but transposed: similarity between item *columns*, predicted rating uses items the user *has* rated. **Item-item beats user-user in practice** because items have more stable rating distributions than users.
7. **Cosine vs Pearson — the only difference is mean-centering.** Pearson is just cosine on mean-centered vectors. Centering removes "harsh-rater" / "lenient-rater" bias and lets *negative* similarity arise (genuinely dissimilar tastes).
8. **Baseline predictor.** Even before any neighbours are consulted: $b_{xi} = \mu + b_x + b_i$ where μ is the global mean rating, $b_x = \bar{} r_x - \mu$ is the user's tendency, $b_i = \bar{} r_i - \mu$ is the item's tendency. Stronger CF combines baseline + neighbour correction: $\hat{} r_{xi} = b_{xi} + frac\Sigma s_{ij}(r_{xj} - b_{xj})\Sigma s_{ij}$.
9. **RMSE is the default metric.** $RMSE = \sqrt{frac1N \Sigma (\hat{} r - r)^2}$ on a held-out test set. Lower is better. RMSE penalises outliers heavily because errors are squared.
10. **Hybrid wins.** Content + collaborative + baseline, often combined linearly. The Netflix-prize-winning system was an ensemble of dozens of such predictors.

The single biggest exam pattern.

The examiner consistently asks: "Given this 4×5 utility matrix, predict user X's rating for item Y using item-item collaborative filtering with *k=2* nearest items, Pearson correlation similarity." Master that pipeline (mean-center → cosine → top-*k* → weighted average) and 60–70% of recsys marks are guaranteed.

§2 — What Is a Recommender System?

The web replaced scarcity with **abundance**: a brick-and-mortar store can stock maybe 3000 movies, Netflix offers tens of thousands; a bookshop carries 5000 titles, Amazon hosts millions. With more choice comes the need for **better filters**. Recommender systems are precisely those filters.

Search vs Recommendation

	Search	Recommendation
User intent	Knows roughly what they want	Open to discovery
Input	Explicit query string	User's history + global trends
Output	Items matching the query	Items the user is *likely* to like
Metric	Precision @ k for the query	RMSE, click-through rate, watch time

Analogy — a friend's recommendation.

When a friend recommends a movie, they implicitly know your taste (your past liked films) and have their own taste (their past liked films). They suggest a film your tastes overlap on. A recommender system formalises this: build a model of you, build a model of the item, score the match.

The Long Tail

Brick-and-mortar shelf space is finite — only the most popular items make the cut. Online stores can carry every item ever made, including *rare* ones each bought by only a few customers. The *aggregate* sales of these rare items can rival the sales of the bestsellers — this is the **long tail** (Anderson, 2004).

Recommender systems are what make the long tail commercially viable: without them, users would never *find* the niche items that match their taste.

Three Worked Examples From the Slides

- **Amazon.** Customer *X* buys Metallica and Megadeth CDs. Customer *Y* searches "Metallica". The system recommends Megadeth to *Y* — leveraging *X*'s purchase pattern.
- **Netflix.** "Because you watched *Inception*…" — content-based recommendation off a single watched item.
- **Google News.** Personalised front page driven by your reading history *and* what users with similar histories click.

Three Generations of Recommenders

Generation	Example	Mechanics
Editorial / curated	"Staff picks"	Hand-built lists
Simple aggregates	"Top 10", "Trending"	Population statistics, no personalisation
Personalised (today)	Amazon, Netflix	Content + collaborative + hybrid models

---

§3 — The Utility Matrix

Formal Setup

Let X be the set of customers and S be the set of items. The utility function $u: X \times S \to R$ maps a (customer, item) pair to a rating. Common rating sets R:

- **Star ratings:** integers *1, 2, 3, 4, 5*\.
- **Like / dislike:** *0, 1*\ or *-1, +1*\.
- **Real number:** *[0, 1]* for predicted purchase probability.

The **utility matrix** U is the table whose row x, column i entry is *u(x, i) — if it exists*. Most entries are missing.

Example — 4 users × 4 movies

	Avatar	LOTR	Matrix	Pirates
Alice	1	—	0.2	—
Bob	—	0.5	—	0.3
Carol	0.2	—	1	—
David	—	—	—	0.4

Out of 16 possible cells, only 7 are filled. This sparsity is the central computational challenge.

Three Problems of a Recommender System

Problem 1 — Gathering ratings. How do we populate U in the first place?

- **Explicit ratings.** Ask the user to rate items (star ratings, thumbs up/down). Accurate but rarely volunteered — most users don't bother.
- **Implicit ratings.** Infer a rating from behaviour. Purchase implies positive; long watch time implies positive; quick close implies negative. Plentiful but noisy.

Problem 2 — Extrapolating unknown ratings. Given the sparse U, predict the empty cells. We mostly care about **high** unknown ratings — there's no point in proving the user *won't* like something they're never going to be shown anyway. Three approaches: content-based (§4), collaborative (§5), latent-factor (out of scope).

Problem 3 — Evaluating extrapolation. How well do our predictions match the true ratings? Hold out some known ratings as a *test set*, compute predictions on them, measure

error (RMSE). Details in §9.

Cold Start

Cold start is the most painful failure mode of any recommender:

- **New item.** No-one has rated it yet; collaborative filtering can't reason about it. Content-based filtering can — feature engineering rescues us.
- **New user.** No history; collaborative filtering has nothing to compare. Workarounds: ask a few onboarding questions, use demographics, default to popularity.

---

§4 — Content-Based Filtering

The Core Idea

Recommend items similar to items the user previously liked.

Recommend items similar to items the user previously liked.

If Alice rated *Inception* 5/5, recommend other Christopher Nolan films / sci-fi thrillers / films starring Leonardo DiCaprio. The recommendation is justified by *features of the items themselves*, not by the behaviour of other users.

Step 1 — Build the Item Profile

For each item, build a feature vector. For movies: **{actors, director, genre, decade, language, …}**. For text articles: **TF-IDF over the document's words**. For songs: **{genre, BPM, key, vocal/instrumental flag, …}**.

TF-IDF Refresher (relevant for text-based items)

Let f_{ij} = raw frequency of term i in document j. Define:

$$\mathrm{TF}_{ij} = \frac{f_{ij}}{\max_k f_{kj}} \qquad \mathrm{IDF}_i = \log \frac{N}{n_i}$$

where n_i is the number of documents containing term i and N is the total number of documents. Then:

$$w_{ij} = \mathrm{TF}_{ij} \cdot \mathrm{IDF}_i$$

The document profile is the sparse vector of words with the highest w_{ij} scores.

EXAM TRAP — TF can be normalised in different ways.

Some sources divide by total words in the document, some by the max-frequency term. Both are valid; check what the question specifies. The spirit is the same: longer documents shouldn't get artificially higher TF.

Step 2 — Build the User Profile

Aggregate the profiles of the items the user has rated. The simplest aggregator is the **weighted average**:

$$\text{userProfile}_x = \frac{\sum_{i \in I_x} r_{xi} \cdot \text{itemProfile}_i}{\sum_{i \in I_x} r_{xi}}$$

where I_x is the set of items rated by user x. A common variant subtracts the user's mean rating before weighting, so items the user rated *unusually highly* have more influence.

Step 3 — Predict via Cosine Similarity

Given a user profile *boldsymbolx* and an item profile *boldsymboli*, predict the user's score for the item as:

$$\hat{u}(x, i) = \cos(\boldsymbol{x}, \boldsymbol{i}) = \frac{\boldsymbol{x} \cdot \boldsymbol{i}}{\|\boldsymbol{x}\| \cdot \|\boldsymbol{i}\|}$$

Cosine ranges over *[-1, +1]* in general, *[0, +1]* when both vectors have non-negative entries (as feature vectors typically do). High cosine ⇒ the user's taste vector points in the same direction as the item's profile ⇒ recommend.

Pros & Cons

Pros	Cons
No cold start for new items — features known at creation	**Feature engineering is hard** — what's a "good" image feature?
Recommends to niche tastes — doesn't depend on a user crowd	**Cold start for new users** — empty user profile
Explainable — "you liked Inception, this is also Nolan"	**Over-specialisation** — never recommends outside known tastes
No first-rater problem — new items can be recommended	**No serendipity** — won't suggest a great film outside genre

EXAM TRAP — content-based vs collaborative cold-start.

Content-based fixes *new-item* cold-start (it can use features) but does NOT fix *new-user* cold-start (still need history). Collaborative fixes neither directly.

---

§5 — Collaborative Filtering (CF)

The Core Idea

Forget item features. Use the rating matrix alone.

"People similar to you also liked X" or "items similar to ones you liked are X". No feature engineering needed.

Two flavours: **User-User CF** and **Item-Item CF**. Identical mathematical machinery; one views the matrix row-wise, the other column-wise.

5.1 User-User CF

Procedure.

1. Compute similarity *sim(x, y)* between user *x* (the target) and every other user *y*.
2. Take *N* = the top-*k* users most similar to *x* who have rated item *i*.
3. Predict ^ *r(x, i)* as the similarity-weighted average of those *k* users' ratings of *i*:

$$\hat{r}(x, i) = \frac{\sum_{y \in N} \text{sim}(x, y) \cdot r(y, i)}{\sum_{y \in N} |\text{sim}(x, y)|}$$

The denominator uses **|sim|** not just sim because Pearson (and centred cosine) similarities can be negative — without absolute values, a negative similarity could blow up the prediction.

EXAM TRAP — denominator is $\sum |s|$, not $\sum s$.

If you forget the absolute value bars and one neighbour has *sim(x, y) = -0.4* and another has *sim(x, y) = +0.4*, the denominator will be exactly zero and your prediction will be undefined. Use absolute values.

Similarity Choices

Let r_x = vector of user *x*'s ratings (with missing entries handled differently per measure).

Jaccard similarity. Treat the rating vectors as **sets** — which items the user rated, ignoring values.

$$J(x, y) = \frac{|S_x \cap S_y|}{|S_x \cup S_y|}$$

Problem: throws away the magnitudes of ratings. A user who rated *X* 1-star and one who rated *X* 5-stars look identical to Jaccard.

Cosine similarity (raw). Treat missing entries as *0*, then:

$$\cos(x, y) = \frac{\sum_i r_{xi} \cdot r_{yi}}{\sqrt{\sum_i r_{xi}^2} \sqrt{\sum_i r_{yi}^2}}$$

Problem: treats "did not rate" the same as "rated 0" — strong negative implied where the user simply hasn't seen the item. This biases similarity downward.

Pearson correlation. Subtract each user's mean rating *before* taking cosine. Operates only on **co-rated items** S_{xy}:

$$\text{sim}(x, y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)(r_{ys} - \bar{r}_y)}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \bar{r}_y)^2}}$$

Pearson handles "harsh raters" vs "lenient raters" cleanly: an Alice who rates 5-stars liberally and a Bob who rates conservatively can still have *sim(Alice, Bob) = +1* if their *deviations from their own means* track each other.

EXAM TRAP — cosine vs Pearson differ ONLY in mean-centering.

Pearson IS centred cosine. If you've already mean-centred both vectors, computing cosine on them gives the Pearson value. The slides explicitly note: "*Notice cosine sim. is correlation when data is centered at 0.*"

5.2 Item-Item CF

Same idea, transposed.

1. For target user *x* and target item *i*, compute similarity *sim(i, j)* between item *i* and every other item *j* that user *x* HAS rated. Similarity is computed on the rating *columns*.
2. Take *N(i; x)* = top-*k* items most similar to *i* that *x* rated.
3. Predict:

$$\hat{r}(x, i) = \frac{\sum_{j \in N(i;x)} \text{sim}(i, j) \cdot r_{xj}}{\sum_{j \in N(i;x)} |\text{sim}(i, j)|}$$

Why Item-Item Usually Beats User-User

Note

"*In practice, it has been observed that item-item often works better than user-user. Why? Items are simpler, users have multiple tastes.*" — slides

Three reasons:

1. **More ratings per item than per user.** Popular items have hundreds of ratings; most users have rated only a few items. Item similarity is statistically more stable.
2. **Item characters are constant.** A movie is what it is; a user's mood changes day to day. Item-item exploits a cleaner signal.
3. **Pre-computability.** Item-item similarities can be cached because the item set changes slowly. User-user similarities shift every time any user adds a rating.

EXAM TRAP — N(i; x) is items rated by user x.

When applying the item-item formula, the neighbour set is *only* items the target user has already rated. You can't include item *j* as a neighbour if user *x* never rated *j* — there's no r_{xj} to plug in.

---

§6 — Implementing Cosine Similarity Properly

The slides make a subtle but exam-relevant point. Consider three users A, B, C with the rating matrix:

	Item 1	Item 2	Item 3	Item 4	Item 5
A	4	—	—	5	1
B	5	5	4	—	—
C	—	—	—	2	4

Intuitively A and B agree on Item 1 (both rated 4 or 5) so we want *sim(A, B) > sim(A, C)*.

Jaccard. *J(A, B) = |\1\|/|\1,2,3,4,5\| = 1/5*. *J(A, C) = |\4, 5\|/|\1,4,5\| = 2/3* from C's perspective, or 2/4 = 1/2 if we use union of all rated. Either way Jaccard says A and C are *more* similar — wrong intuition, because Jaccard ignores values.

Raw cosine (treat missing = 0). *A = (4, 0, 0, 5, 1)*, *B = (5, 5, 4, 0, 0)*, *C = (0, 0, 0, 2, 4)*.

$$\cos(A, B) = \frac{4 \cdot 5 + 0 + 0 + 0 + 0}{\sqrt{42} \cdot \sqrt{66}} \approx \frac{20}{52.6} \approx 0.380$$

$$\cos(A, C) = \frac{0 + 0 + 0 + 5 \cdot 2 + 1 \cdot 4}{\sqrt{42} \cdot \sqrt{20}} \approx \frac{14}{28.98} \approx 0.483$$

So raw cosine *also* says A is more similar to C than to B — also "wrong" because the raw cosine treats missing entries as negative information.

Mean-centred cosine (= Pearson). Subtract each row's mean before taking cosine.

- $\bar{r}_A$ *= (4 + 5 + 1)/3 = 3.33*. Centred A on items rated: *(0.67, 0, 0, 1.67, -2.33)* — leave 0 for unrated entries.
- $\bar{r}_B$ *= (5 + 5 + 4)/3 = 4.67*. Centred B: *(0.33, 0.33, -0.67, 0, 0)*.
- $\bar{r}_C$ *= (2 + 4)/2 = 3*. Centred C: *(0, 0, 0, -1, 1)*.

Recompute cosine on the centred vectors:

$$\cos(A_c, B_c) = \frac{0.67 \cdot 0.33 + 0 + 0 + 0 + 0}{\|A_c\| \cdot \|B_c\|} \approx 0.092$$

$$\cos(A_c, C_c) = \frac{0 + 0 + 0 + 1.67 \cdot (-1) + (-2.33) \cdot 1}{\|A_c\| \cdot \|C_c\|} \approx -0.559$$

Now the *correct* intuition emerges: *sim(A, B) > sim(A, C)*, and *sim(A, C)* is genuinely *negative* — A and C have opposite tastes on the items they both rated.

TAKEAWAY.

Always mean-centre before computing cosine for collaborative filtering. The non-centred cosine is mathematically valid but gives misleading similarities because of the "missing = 0" interpretation problem.

§7 — Baseline Predictors

Even before any neighbour is consulted, we can produce a reasonable rating estimate from population statistics alone.

The Three Components

- μ = global mean rating across all (user, item) pairs in the training data.

- $b_x = \bar{r}_x - \mu$ = user x's deviation from the global mean. Positive for lenient raters, negative for harsh ones.
- $b_i = \bar{r}_i - \mu$ = item i's deviation from the global mean. Positive for highly-rated items, negative for poorly-rated ones.

The **baseline estimate** for user x's rating of item i:

$$b_{xi} = \mu + b_x + b_i$$

Slides Example — Joe and "The Sixth Sense"

- Mean movie rating $\mu = 3.7$ stars.
- *The Sixth Sense* averages *0.5* stars above the global mean: $b_{Sixth\ Sense} = +0.5$.
- Joe rates *0.2* stars below the global mean: $b_{Joe} = -0.2$.

Baseline estimate:

$$b_{\text{Joe, Sixth Sense}} = 3.7 + (-0.2) + 0.5 = 4.0 \text{ stars}$$

Joe will give The Sixth Sense 4 stars

even though he hasn't rated any "similar" movie that collaborative filtering could lean on. The baseline gets us there from population stats alone.

Combining Baseline with Item-Item CF

The full formula is:

$$\hat{r}_{xi} = b_{xi} + \frac{\sum_{j \in N(i;x)} s_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i;x)} |s_{ij}|}$$

The neighbour term predicts the *deviation* from baseline rather than the absolute rating. This is more accurate than naive item-item because the baseline already absorbs user-bias and item-bias variance.

Slides example continued. Baseline says Joe will rate *The Sixth Sense* 4 stars. But the only similar movie Joe rated is *Signs*, which he scored 1 star *below* his average. Then:

$$\hat{r}_{\text{Joe, Sixth Sense}} = 4 + (-1) = 3 \text{ stars}$$

The baseline + neighbour-deviation framework is exactly what the Netflix-prize-winning systems used.

---

§8 — Hybrid Approaches

No single method dominates. Production systems combine:

1. **Content-based predictor** ^ $r^{(C)}_{xi}$ — handles new items.
2. **Collaborative predictor** ^ $r^{(CF)}_{xi}$ — handles taste discovery.
3. **Baseline predictor** b_{xi} — handles the "no signal yet" case.

A common combination is a **linear ensemble**:

$$\hat{r}_{xi} = w_1 \cdot \hat{r}_{xi}^{(C)} + w_2 \cdot \hat{r}_{xi}^{(\mathrm{CF})} + w_3 \cdot b_{xi}$$

where the weights w_1, w_2, w_3 are tuned on a validation set (typically by gradient descent minimising RMSE).

Demographic & Content Add-ons for Cold-Start

- **New user → use demographics.** Age, region, sign-up source. Predict from cohort behaviour until enough ratings accumulate.
- **New item → use content profile.** Until enough users rate, recommend based on item features.

This is exactly the Netflix prize lesson: **the winning entry was an ensemble of dozens of models** (k-NN at multiple *k*, latent-factor models, restricted Boltzmann machines, regression on side data). No single model wins — the ensemble does.

---

§9 — Evaluation

Holdout Test Set

Split your known ratings randomly into a training set (used to learn predictions) and a test set (the "ground truth" the predictor must match). Common splits: 80/20 or 90/10.

RMSE — Root Mean Squared Error

$$\mathrm{RMSE} = \sqrt{\frac{1}{N} \sum_{(x,\, i) \in \mathrm{Test}} (\hat{r}_{xi} - r^*_{xi})^2}$$

where r^*_{xi} is the held-out true rating and N is the number of test pairs. Lower is better.

EXAM TRAP — RMSE squares errors so penalises outliers more than MAE.

A single prediction off by *3* contributes *9* to the sum, while three predictions off by *1* each contribute only *3* total. RMSE is *not* robust to a few terrible predictions.

Other Metrics Beyond RMSE

Metric	What it measures	When useful
Precision @ k	Fraction of top-*k* recommendations the user actually liked	Browsing UX where only top picks matter
Recall @ k	Fraction of liked items captured in top-*k*	Coverage of user preferences
Spearman's rank	Correlation of system ranking vs user's true ranking	Order matters more than absolute ratings
ROC / AUC	Tradeoff false positives vs false negatives	Binary like/dislike systems

Metric	What it measures	When useful
Coverage	Fraction of (user, item) pairs the system can predict at all	Cold-start severity
Diversity	How varied the recommendations are	Avoiding filter bubbles

The "Care About High Ratings" Caveat

> **Note**
>
> "*In practice, we care only to predict high ratings: RMSE might penalize a method that does well for high ratings and badly for others.*" — slides

If your goal is to surface the user's top picks, errors on items they'd rate 1-star don't hurt — those items would never be shown. RMSE doesn't know this, so production teams often weight test errors by predicted rating, or use rank-based metrics (Precision@10, NDCG) instead.

Speed of Recommendation

The expensive step in CF is finding the k most similar users (or items): naive cost $O(|X|)$ per query. We have already met faster alternatives:

- **LSH** (Week 4–5 — fast similarity hashing).
- **Clustering** — compute similarities only within cluster.
- **Dimensionality reduction** — SVD / latent factors.

---

§10 — Six Worked Numerical Examples

These are the **most important** examples in this PDF. Read each once; close the document; reproduce on paper from scratch. The exam will ask one of these in some form.

Worked Example 1 — Pearson Similarity Between Two Users

Problem. Users Alice and Bob rate 5 movies. Compute the Pearson similarity sim(Alice, Bob) by hand.

Movie	M1	M2	M3	M4	M5
Alice	5	3	4	4	1
Bob	3	1	2	3	5

Step 1 — co-rated items. Both users rated all five movies, so S_{AB} = *M1, M2, M3, M4, M5*\.

Step 2 — means.

- $\bar{r}_A = (5 + 3 + 4 + 4 + 1)/5 = 17/5 = 3.4$.
- $\bar{r}_B = (3 + 1 + 2 + 3 + 5)/5 = 14/5 = 2.8$.

Step 3 — centred vectors.

- $A_c = (5\text{-}3.4, 3\text{-}3.4, 4\text{-}3.4, 4\text{-}3.4, 1\text{-}3.4) = (1.6, -0.4, 0.6, 0.6, -2.4)$.

- B_c = *(3-2.8, 1-2.8, 2-2.8, 3-2.8, 5-2.8) = (0.2, -1.8, -0.8, 0.2, 2.2)*.

Step 4 — numerator (dot product of centred vectors).

$$\sum(r_{As} - \bar{r}_A)(r_{Bs} - \bar{r}_B) = 1.6 \cdot 0.2 + (-0.4)(-1.8) + 0.6(-0.8) + 0.6 \cdot 0.2 + (-2.4)(2.2)$$
$$= 0.32 + 0.72 - 0.48 + 0.12 - 5.28 = -4.60$$

Step 5 — denominators (norms of centred vectors).

$$r_{As} - \bar{r}_A)^2 = 1.6^2 + 0.4^2 + 0.6^2 + 0.6^2 + 2.4^2 = 2.56 + 0.16 + 0.36 + 0.36 + 5.76 = 9$$
$$r_{Bs} - \bar{r}_B)^2 = 0.2^2 + 1.8^2 + 0.8^2 + 0.2^2 + 2.2^2 = 0.04 + 3.24 + 0.64 + 0.04 + 4.84 = 8$$
$$\sqrt{9.20} \approx 3.033, \qquad \sqrt{8.80} \approx 2.966$$

Step 6 — Pearson.

$$\text{sim}(A, B) = \frac{-4.60}{3.033 \cdot 2.966} = \frac{-4.60}{8.998} \approx -0.511$$

Interpretation. Alice and Bob have *negative* correlation — Alice's high movies are Bob's low movies and vice versa. They are anti-similar.

Worked Example 2 — User-User CF Rating Prediction

Problem. Given the utility matrix below, predict Alice's rating of M_3 using user-user CF with $k = 2$ nearest users by Pearson similarity.

User	M1	M2	M3	M4	M5
Alice	5	3	?	4	1
Bob	3	1	2	3	5
Carol	4	3	4	3	2
David	3	3	1	5	4

Step 1 — Pearson similarities of Alice with each other user. (We exclude M_3 from each calculation because Alice didn't rate it — only co-rated items count.)

Alice's mean over the **4 movies she rated** = *(5+3+4+1)/4 = 3.25*.

Alice vs Bob (over M_1, M_2, M_4, M_5):

- Bob's mean over those 4 movies = *(3+1+3+5)/4 = 3.0*.
- A_c = *(5-3.25, 3-3.25, 4-3.25, 1-3.25) = (1.75, -0.25, 0.75, -2.25)*.
- B_c = *(3-3.0, 1-3.0, 3-3.0, 5-3.0) = (0, -2, 0, 2)*.
- Numerator: *1.75 · 0 + (-0.25)(-2) + 0.75 · 0 + (-2.25)(2) = 0 + 0.5 + 0 - 4.5 = -4.0*.
- $\|A_c\|^2$ = *3.0625 + 0.0625 + 0.5625 + 5.0625 = 8.75*, so $\|A_c\|$ = *2.958*.
- $\|B_c\|^2$ = *0 + 4 + 0 + 4 = 8*, so $\|B_c\|$ = *2.828*.
- *sim(A, B) = -4.0 / (2.958 · 2.828) = -4.0 / 8.366 ≈ -0.478*.

Alice vs Carol (over M_1, M_2, M_4, M_5):

- Carol's mean = *(4+3+3+2)/4 = 3.0*.
- C_c = *(4-3, 3-3, 3-3, 2-3) = (1, 0, 0, -1)*.

- Numerator: $1.75 \cdot 1 + (-0.25)(0) + 0.75 \cdot 0 + (-2.25)(-1) = 1.75 + 0 + 0 + 2.25 = 4.00$.
- $\|C_c\|^2 = 1 + 0 + 0 + 1 = 2$, so $\|C_c\| = 1.414$.
- $sim(A, C) = 4.0 / (2.958 \cdot 1.414) = 4.0 / 4.183 \approx +0.956$.

Alice vs David (over M_1, M_2, M_4, M_5):

- David's mean = $(3+3+5+4)/4 = 3.75$.
- $D_c = (3-3.75, 3-3.75, 5-3.75, 4-3.75) = (-0.75, -0.75, 1.25, 0.25)$.
- Numerator: $1.75 \cdot (-0.75) + (-0.25)(-0.75) + 0.75 \cdot 1.25 + (-2.25)(0.25)$

$= -1.3125 + 0.1875 + 0.9375 - 0.5625 = -0.75$.

- $\|D_c\|^2 = 0.5625 + 0.5625 + 1.5625 + 0.0625 = 2.75$, so $\|D_c\| = 1.658$.
- $sim(A, D) = -0.75 / (2.958 \cdot 1.658) = -0.75 / 4.904 \approx -0.153$.

Summary of similarities: Carol $+0.956$, David -0.153, Bob -0.478.

Step 2 — pick top-k = 2 neighbours by similarity. By **absolute** value: Carol (0.956) and Bob (0.478) are the top two.

Step 3 — predict using the weighted-average formula.

$$\hat{r}(\text{Alice}, M_3) = \frac{\text{sim}(A, C) \cdot r(C, M_3) + \text{sim}(A, B) \cdot r(B, M_3)}{|\text{sim}(A, C)| + |\text{sim}(A, B)|}$$

$$= \frac{0.956 \cdot 4 + (-0.478) \cdot 2}{0.956 + 0.478} = \frac{3.824 - 0.956}{1.434} = \frac{2.868}{1.434} \approx 2.00$$

Interpretation. Alice would be predicted to rate M_3 at ≈ 2 stars — Carol pulls her up (Carol liked it, Alice and Carol agree), Bob pulls her down (Bob liked it less, and Alice and Bob disagree).

Worked Example 3 — Item-Item CF Prediction on the Same Matrix

Problem. Same utility matrix. Now predict Alice's rating for M_3 using **item-item** CF with $k = 2$ nearest items.

Step 1 — Pearson similarity of M_3 with every other movie. Compute on the *column* of each movie. Use only users who rated *both* movies.

Let's mean-centre each column (using each movie's mean over the users who rated it):

Movie	Ratings vector (Bob, Carol, David)	Mean	Centred
M_3	(2, 4, 1)	7/3 ≈ 2.33	(-0.33, 1.67, -1.33)
M_1	(3, 4, 3) over (Bob, Carol, David)	10/3 ≈ 3.33	(-0.33, 0.67, -0.33)
M_2	(1, 3, 3)	7/3 ≈ 2.33	(-1.33, 0.67, 0.67)
M_4	(3, 3, 5)	11/3 ≈ 3.67	(-0.67, -0.67, 1.33)
M_5	(5, 2, 4)	11/3 ≈ 3.67	(1.33, -1.67, 0.33)

(Only Bob, Carol, David rated M_3, so we restrict similarity computation to those three users for every column.)

$sim(M_3, M_1)$:

- Numerator: *(-0.33)(-0.33) + (1.67)(0.67) + (-1.33)(-0.33) = 0.109 + 1.119 + 0.439 = 1.667*.
- $\|M_3\|^2 = 0.109 + 2.789 + 1.769 = 4.667$, $\|M_3\| = 2.160$.
- $\|M_1\|^2 = 0.109 + 0.449 + 0.109 = 0.667$, $\|M_1\| = 0.816$.
- sim = *1.667 / (2.160 · 0.816) = 1.667 / 1.763 ≈ +0.946*.

sim(M_3, M_2):

- Numerator: *(-0.33)(-1.33) + (1.67)(0.67) + (-1.33)(0.67) = 0.439 + 1.119 - 0.891 = 0.667*.
- $\|M_2\|^2 = 1.769 + 0.449 + 0.449 = 2.667$, $\|M_2\| = 1.633$.
- sim = *0.667 / (2.160 · 1.633) = 0.667 / 3.527 ≈ +0.189*.

sim(M_3, M_4):

- Numerator: *(-0.33)(-0.67) + (1.67)(-0.67) + (-1.33)(1.33) = 0.221 - 1.119 - 1.769 = -2.667*.
- $\|M_4\|^2 = 0.449 + 0.449 + 1.769 = 2.667$, $\|M_4\| = 1.633$.
- sim = *-2.667 / (2.160 · 1.633) = -0.756*.

sim(M_3, M_5):

- Numerator: *(-0.33)(1.33) + (1.67)(-1.67) + (-1.33)(0.33) = -0.439 - 2.789 - 0.439 = -3.667*.
- $\|M_5\|^2 = 1.769 + 2.789 + 0.109 = 4.667$, $\|M_5\| = 2.160$.
- sim = *-3.667 / (2.160 · 2.160) = -3.667 / 4.666 ≈ -0.786*.

Step 2 — top-*k* = *2* items most similar to M_3 that Alice rated. Alice rated M_1, M_2, M_4, M_5. By absolute similarity: M_1 (*0.946*), M_5 (*0.786*).

Step 3 — predict. Alice's ratings: *r(Alice, M_1) = 5*, *r(Alice, M_5) = 1*.

$$\hat{r}(\text{Alice}, M_3) = \frac{0.946 \cdot 5 + (-0.786) \cdot 1}{0.946 + 0.786} = \frac{4.730 - 0.786}{1.732} = \frac{3.944}{1.732} \approx 2.28$$

Interpretation. Item-item gives ≈ 2.28 vs user-user's 2.00 — the same ballpark, slightly different because the methods weight different evidence. In practice both are reasonable answers; the exam will award marks for either as long as the procedure is correct.

Worked Example 4 — Raw Cosine Similarity (No Centering)

Problem. Compute the raw cosine similarity between item columns M_1 and M_3. Treat missing entries as *0* (the problematic interpretation, but the slides explicitly compare).

M_1 column: *(5, 3, 4, 3)* = (Alice, Bob, Carol, David). M_3 column: *(0, 2, 4, 1)* — Alice's missing, treated as *0*.

Step 1 — dot product.

$$M_1 \cdot M_3 = 5 \cdot 0 + 3 \cdot 2 + 4 \cdot 4 + 3 \cdot 1 = 0 + 6 + 16 + 3 = 25$$

Step 2 — norms.

$$\|M_1\| = \sqrt{25 + 9 + 16 + 9} = \sqrt{59} \approx 7.681$$

$$\|M_3\| = \sqrt{0 + 4 + 16 + 1} = \sqrt{21} \approx 4.583$$

Step 3 — cosine.

$$\cos(M_1, M_3) = \frac{25}{7.681 \cdot 4.583} = \frac{25}{35.20} \approx 0.710$$

Compare to centred (Pearson) version above: ≈ *+0.946*. The centred version reflects co-movement of *deviations* whereas the raw cosine is dragged toward 0 by the implicit "Alice rated M_3 as 0" assumption. Always prefer centred for CF.

Worked Example 5 — Baseline Predictor

Problem. Compute the baseline rating that Alice will assign to The Sixth Sense, given:

- Global mean rating $\mu = 3.7$.
- Alice's average rating is *0.4* stars *above* the global mean.
- The Sixth Sense's average rating is *0.5* stars *above* the global mean.

Step 1 — identify the bias terms.

- $b_{Alice} = +0.4$.
- $b_{Sixth\ Sense} = +0.5$.

Step 2 — apply the baseline formula.

$$b_{\text{Alice, Sixth Sense}} = \mu + b_{\text{Alice}} + b_{\text{Sixth Sense}} = 3.7 + 0.4 + 0.5 = 4.6$$

Interpretation. Alice will rate The Sixth Sense ≈ *4.6* stars on the global-baseline-only model. If we then incorporate her actual neighbour ratings, the prediction may shift up or down — but the baseline is the starting point.

EXAM TRAP — sign of the bias.

b_x is *user mean minus global mean*, so it can be negative for harsh raters. The slides use Joe at $b_{Joe} = -0.2$. Don't always assume positive.

Worked Example 6 — RMSE on a Held-Out Set

Problem. A recommender produces these 5 (predicted, actual) pairs on the test set. Compute RMSE.

Pair	Predicted	Actual
1	4.0	5
2	3.5	3
3	2.0	1
4	4.5	4
5	3.0	4

Step 1 — squared errors.

Pair	^ r - r^*	(^ r - r^*)^2
1	*4.0 - 5 = -1.0*	*1.00*
2	*3.5 - 3 = +0.5*	*0.25*

Pair	^ r - r^*	(^ r - r^*)^2
3	*2.0 - 1 = +1.0*	*1.00*
4	*4.5 - 4 = +0.5*	*0.25*
5	*3.0 - 4 = -1.0*	*1.00*
Sum		*3.50*

Step 2 — divide by N = 5 and square-root.

$$\text{RMSE} = \sqrt{\frac{3.50}{5}} = \sqrt{0.70} \approx 0.837$$

Interpretation. On average the predictor is off by about *0.84* stars. For 5-star ratings, that's "okay" — Netflix-prize-winning systems achieved RMSE ≈ 0.86–0.88 on the Netflix dataset.

EXAM TRAP — RMSE vs MAE.

MAE = average of *absolute* errors = *(1 + 0.5 + 1 + 0.5 + 1)/5 = 0.80*. MAE here is *less* than RMSE (0.80 vs 0.84) because RMSE squares before averaging, amplifying the larger errors. RMSE > MAE in general.

§11 — Practice Questions (15)

Mix: 6 numerical traces, 5 conceptual short-answer, 4 MCQ / true-false. Time yourself: ≈ 7 min per numerical, ≈ 3 min per conceptual, ≈ 1 min per MCQ.

Q1 [Numerical · 7 marks] ** Three users U1, U2, U3 rate four movies:

User	A	B	C	D
U1	4	3	5	2
U2	5	4	4	3
U3	1	5	2	4

(a) Compute $\bar{r}_{U1}$, $\bar{r}_{U2}$, $\bar{r}_{U3}$. (b) Compute Pearson sim(U1, U2) and sim(U1, U3) over all four movies. (c) Identify U1's most similar user.

Q2 [Numerical · 8 marks] ** Using the matrix from §10 WE2 (Alice/Bob/Carol/David), predict David's rating for M_3 using user-user CF with $k = 2$ Pearson neighbours. Show every step. (Alice's M_3 is unknown — exclude her from David's neighbour search.)

Q3 [Numerical · 8 marks] ** Same matrix as WE2/WE3. Predict Bob's rating of M_5? — wait, Bob already rated M_5. Predict Carol's rating of M_2 using item-item CF with $k = 2$ Pearson neighbours. (Use only users who rated both compared items for each pair.)

Q4 [Numerical · 5 marks] ** Compute the **raw cosine** similarity (no centering, missing = 0) between item columns A and B from Q1.

Q5 [Numerical · 6 marks] ** Global mean $\mu = 3.6$. User Bob has average rating 3.2. The movie *Inception* has an average rating 4.1. Bob's neighbour Sam rated *Inception* 5 and rated the most similar item *Interstellar* with sim($\text{Inception}, \text{Interstellar}$) $= 0.7$ giving $r_{\text{Bob, Interstellar}} = 4$ (which is 0.5 above Bob's average — so deviation $= +0.5$). Use the **baseline + neighbour-deviation** formula with the single neighbour Interstellar to predict Bob's rating of Inception.

Q6 [Numerical · 6 marks] ** A recommender produces these test-set (predicted, actual) pairs: $(4.5, 5), (3.0, 4), (2.5, 1), (4.0, 4), (1.5, 2), (5.0, 4)$. Compute (a) MAE and (b) RMSE. Comment on which is larger and why.

Q7 [Concept · 4 marks] ** State the three problems any recommender system must solve. For each, give one specific example of a technique used to address it.

Q8 [Concept · 4 marks] ** Explain in 4–6 sentences why item-item collaborative filtering typically outperforms user-user. Mention at least two distinct reasons.

Q9 [Concept · 4 marks] ** Compare cold-start behaviour of content-based vs collaborative filtering. Specifically: (a) new item, (b) new user, (c) sparse matrix overall. Which method handles which case better?

Q10 [Concept · 3 marks] ** Why must we use $\sum |s_{ij}|$ (absolute values) in the denominator of the rating-prediction formula instead of $\sum s_{ij}$? Give a small numerical example showing what goes wrong.

Q11 [Concept · 3 marks] ** Why is Pearson correlation effectively "centred cosine"? Show algebraically how the two are related.

Q12 [MCQ · 2 marks] ** For the rating prediction formula $\hat r(x, i) = \frac{\sum_y s_{xy} r_{yi}}{\sum_y |s_{xy}|}$, the neighbours y in the sum are: (A) all users in the system (B) all users similar to x, regardless of whether they rated i (C) the k users most similar to x who have rated i (D) the k users most similar to x, with $r_{yi} = 0$ if y didn't rate i

Q13 [MCQ · 2 marks] ** Which is FALSE about RMSE? (A) RMSE is non-negative (B) RMSE = 0 implies perfect prediction (C) RMSE penalises outliers more than MAE (D) RMSE is always equal to or less than MAE

Q14 [True/False · 2 marks] ** Content-based filtering can recommend completely new items that no user has rated yet, while collaborative filtering cannot. Justify.

Q15 [MCQ · 2 marks] ** A recommender computes user-user similarity via Pearson and gets $\text{sim}(A, B) = -0.6$, $\text{sim}(A, C) = +0.4$, $\text{sim}(A, D) = +0.3$. With $k = 2$ neighbours and the standard prediction formula, who are A's neighbours for predicting a rating? (A) C and D (highest positive similarities) (B) B and C (largest absolute similarities) (C) B alone (most negative) (D) all three with weights normalized

---

§12 — Full Worked Answers

A1. (a)** $\bar{r}_{U1} = (4+3+5+2)/4 = 14/4 = 3.5$. $\bar{r}_{U2} = (5+4+4+3)/4 = 16/4 = 4.0$. $\bar{r}_{U3} = (1+5+2+4)/4 = 12/4 = 3.0$.

(b) Pearson sim(U1, U2). Centred:

- $U1_c = (0.5, -0.5, 1.5, -1.5)$.

- $U2_c = (1.0, 0, 0, -1.0)$.
- Numerator: $0.5(1) + (-0.5)(0) + 1.5(0) + (-1.5)(-1) = 0.5 + 0 + 0 + 1.5 = 2.0$.
- $\|U1_c\|^2 = 0.25 + 0.25 + 2.25 + 2.25 = 5.0$, $\|U1_c\| = 2.236$.
- $\|U2_c\|^2 = 1 + 0 + 0 + 1 = 2.0$, $\|U2_c\| = 1.414$.
- sim(U1, U2) $= 2.0 / (2.236 \cdot 1.414) = 2.0 / 3.162 \approx +0.632$.

Pearson sim(U1, U3). Centred:

- $U1_c = (0.5, -0.5, 1.5, -1.5)$ as above.
- $U3_c = (-2, 2, -1, 1)$.
- Numerator: $0.5(-2) + (-0.5)(2) + 1.5(-1) + (-1.5)(1) = -1 -1 - 1.5 - 1.5 = -5.0$.
- $\|U3_c\|^2 = 4 + 4 + 1 + 1 = 10$, $\|U3_c\| = 3.162$.
- sim(U1, U3) $= -5.0 / (2.236 \cdot 3.162) = -5.0 / 7.071 \approx -0.707$.

(c) U1's most similar user is **U2** (sim $+0.632$ is more positive than U3's -0.707). Interpretation: U2's tastes track U1's; U3's are anti-correlated.

A2. ** Predict David's rating of M_3.

David rated all of M_1, M_2, M_3, M_4, M_5 — wait, the problem asks to predict M_3, so we need to *re-do* with M_3 unknown for David. **Re-reading the problem:** the matrix in WE2 had Alice's M_3 unknown; now we hide David's M_3 instead. David's known ratings: $M_1=3$, $M_2=3$, $M_4=5$, $M_5=4$.

Step 1 — David's mean over the 4 known movies $= (3+3+5+4)/4 = 3.75$.

Step 2 — Pearson similarity of David with Bob, Carol (excluding Alice because she didn't rate M_3 either, so her info doesn't help us here — actually she COULD because we're predicting David's M_3 and she's another user, but she also lacks M_3, so include only users who rated M_3: that's Bob and Carol).

David vs Bob over $\{M_1, M_2, M_4, M_5\}$:

- Bob's mean over those 4 $= (3+1+3+5)/4 = 3.0$.
- $D_c = (3-3.75, 3-3.75, 5-3.75, 4-3.75) = (-0.75, -0.75, 1.25, 0.25)$.
- $B_c = (0, -2, 0, 2)$.
- Numerator: $(-0.75)(0) + (-0.75)(-2) + (1.25)(0) + (0.25)(2) = 0 + 1.5 + 0 + 0.5 = 2.0$.
- $\|D_c\|^2 = 0.5625 + 0.5625 + 1.5625 + 0.0625 = 2.75$, $\|D_c\| = 1.658$.
- $\|B_c\|^2 = 8$, $\|B_c\| = 2.828$.
- sim(D, B) $= 2.0 / (1.658 \cdot 2.828) = 2.0 / 4.689 \approx +0.427$.

David vs Carol over $\{M_1, M_2, M_4, M_5\}$:

- Carol's mean $= (4+3+3+2)/4 = 3.0$.
- $C_c = (1, 0, 0, -1)$.
- Numerator: $(-0.75)(1) + (-0.75)(0) + (1.25)(0) + (0.25)(-1) = -0.75 + 0 + 0 - 0.25 = -1.0$.
- $\|C_c\|^2 = 2$, $\|C_c\| = 1.414$.
- sim(D, C) $= -1.0 / (1.658 \cdot 1.414) = -1.0 / 2.345 \approx -0.426$.

Step 3 — top-2 by absolute similarity: Bob (0.427) and Carol (0.426). Both made the cut.

Step 4 — predict.

$$\hat{r}(\text{David}, M_3) = \frac{0.427 \cdot 2 + (-0.426) \cdot 4}{0.427 + 0.426} = \frac{0.854 - 1.704}{0.853} = \frac{-0.850}{0.853} \approx -1.00$$

That's negative — outside the rating scale. This often happens with small datasets and tight similarities of opposite sign. In practice we'd clamp to the rating range *[1, 5]* giving ***^r ≈ 1***. The exam will accept either: the raw formula output (≈ *-1.0*) with a comment that it would be clamped, or the clamped value.

A3. ** Predict Carol's rating of M_2 using item-item CF.

Carol's known ratings: $M_1 = 4$, $M_3 = 4$, $M_4 = 3$, $M_5 = 2$.

We need similarity of M_2 with each of Carol's rated items, computed over users who rated both.

sim(M_2, M_1) over \Alice, Bob, Carol, David\ (everyone rated both):

- M_2 values: (3, 1, 3, 3). Mean = 2.5. Centred: *(0.5, -1.5, 0.5, 0.5)*.
- M_1 values: (5, 3, 4, 3). Mean = 3.75. Centred: *(1.25, -0.75, 0.25, -0.75)*.
- Numerator: *0.5(1.25) + (-1.5)(-0.75) + (0.5)(0.25) + (0.5)(-0.75) = 0.625 + 1.125 + 0.125 - 0.375 = 1.5*.
- $\|M_2\|^2 = 0.25 + 2.25 + 0.25 + 0.25 = 3.0$, $\|M_2\| = 1.732$.
- $\|M_1\|^2 = 1.5625 + 0.5625 + 0.0625 + 0.5625 = 2.75$, $\|M_1\| = 1.658$.
- sim = *1.5 / (1.732 · 1.658) = 1.5 / 2.872 ≈ +0.522*.

sim(M_2, M_3) over \Bob, Carol, David\ (only those rated M_3):

- M_2 on those users: (1, 3, 3). Mean = 7/3 ≈ 2.33. Centred: *(-1.33, 0.67, 0.67)*.
- M_3 on those users: (2, 4, 1). Mean = 7/3 ≈ 2.33. Centred: *(-0.33, 1.67, -1.33)*.
- Numerator: *(-1.33)(-0.33) + (0.67)(1.67) + (0.67)(-1.33) = 0.439 + 1.119 - 0.891 = 0.667*.
- $\|M_2\|^2 = 1.769 + 0.449 + 0.449 = 2.667$, $\|M_2\| = 1.633$.
- $\|M_3\|^2 = 0.109 + 2.789 + 1.769 = 4.667$, $\|M_3\| = 2.160$.
- sim = *0.667 / (1.633 · 2.160) = 0.667 / 3.527 ≈ +0.189*.

sim(M_2, M_4) over \Alice, Bob, Carol, David\:

- M_2 values: (3, 1, 3, 3). Mean = 2.5. Centred: *(0.5, -1.5, 0.5, 0.5)*.
- M_4 values: (4, 3, 3, 5). Mean = 3.75. Centred: *(0.25, -0.75, -0.75, 1.25)*.
- Numerator: *0.5(0.25) + (-1.5)(-0.75) + (0.5)(-0.75) + (0.5)(1.25) = 0.125 + 1.125 - 0.375 + 0.625 = 1.5*.
- $\|M_4\|^2 = 0.0625 + 0.5625 + 0.5625 + 1.5625 = 2.75$, $\|M_4\| = 1.658$.
- sim = *1.5 / (1.732 · 1.658) = 1.5 / 2.872 ≈ +0.522*.

sim(M_2, M_5) over \Alice, Bob, Carol, David\:

- M_2 values: (3, 1, 3, 3). Centred: *(0.5, -1.5, 0.5, 0.5)*.
- M_5 values: (1, 5, 2, 4). Mean = 3.0. Centred: *(-2, 2, -1, 1)*.
- Numerator: *(0.5)(-2) + (-1.5)(2) + (0.5)(-1) + (0.5)(1) = -1 - 3 - 0.5 + 0.5 = -4.0*.
- $\|M_5\|^2 = 4 + 4 + 1 + 1 = 10$, $\|M_5\| = 3.162$.
- sim = *-4.0 / (1.732 · 3.162) = -4.0 / 5.477 ≈ -0.730*.

Top-2 by absolute similarity: M_5 (*0.730*) and M_1 or M_4 tied at *0.522*. Pick M_1 (alphabetical tiebreak in any reasonable convention; some answers might pick M_4 — both are valid).

Predict (using M_5 and M_1):

$$\hat{r}(\text{Carol}, M_2) = \frac{(-0.730)(2) + (0.522)(4)}{0.730 + 0.522} = \frac{-1.460 + 2.088}{1.252} = \frac{0.628}{1.252} \approx 0.50$$

That's again outside the rating scale; clamp to **^ r ≈ 1**. Or, if you used M_4 instead of M_1 for the second neighbour, the prediction would be:

$$\hat{r} = \frac{(-0.730)(2) + (0.522)(3)}{0.730 + 0.522} = \frac{-1.460 + 1.566}{1.252} = \frac{0.106}{1.252} \approx 0.08$$

Either way: very low predicted rating, dominated by the strongly negative similarity of M_5 to M_2 (Carol rated M_5 at 2, that drags her predicted M_2 down).

A4. ** Raw cosine, missing = 0. Treat full columns over all four users.

- *A = (4, 5, 1, 3)* (rated by U1, U2, Carol-equivalent, U3 if we map Q1's matrix). Wait, this question references Q1's matrix. Let's redo with Q1's matrix:
- Item *A* column: *(U1=4, U2=5, U3=1) = (4, 5, 1)*.
- Item *B* column: *(3, 4, 5)*.
- Dot product: *4 · 3 + 5 · 4 + 1 · 5 = 12 + 20 + 5 = 37*.
- \|*A*\| = *√16 + 25 + 1 = √42 ≈ 6.481*.
- \|*B*\| = *√9 + 16 + 25 = √50 ≈ 7.071*.
- *cos(A, B) = 37 / (6.481 · 7.071) = 37 / 45.83 ≈ +0.807*.

A5. ** Baseline + neighbour-deviation prediction.

Step 1 — baseline for Bob, Inception:

- $\mu = 3.6$, $b_{Bob} = 3.2 - 3.6 = -0.4$, $b_{Inception} = 4.1 - 3.6 = +0.5$.
- $b_{Bob,\ Inception} = 3.6 + (-0.4) + 0.5 = 3.7$.

Step 2 — neighbour deviation. Bob's rating of Interstellar was 4. Bob's average is 3.2, so deviation $r_{Bob,\ Interstellar} - b_{Bob,\ Interstellar}$. We need $b_{Bob,\ Interstellar}$. Wait — the problem states the deviation is *+0.5* (above Bob's average). So $r_{xj} - b_{xj} = 0.5$. The single neighbour has sim = *0.7*.

Step 3 — apply formula.

$$\hat{r}_{\text{Bob, Inception}} = b_{xi} + \frac{\sum s_{ij}(r_{xj} - b_{xj})}{\sum |s_{ij}|} = 3.7 + \frac{0.7 \cdot 0.5}{0.7} = 3.7 + 0.5 = 4.2$$

Bob is predicted to rate Inception 4.2 stars.

A6. ** Errors:

Pair	^ r - r^*	$	err	$	err^2
1	*-0.5*	*0.5*	*0.25*		
2	*-1.0*	*1.0*	*1.00*		
3	*+1.5*	*1.5*	*2.25*		

Pair	^ r - r^*	$	err	$	err^2
4	*0*	*0*	*0*		
5	*-0.5*	*0.5*	*0.25*		
6	*+1.0*	*1.0*	*1.00*		

(a) *MAE = (0.5 + 1.0 + 1.5 + 0 + 0.5 + 1.0)/6 = 4.5 / 6 = 0.75*.

(b) *RMSE = √(0.25 + 1.00 + 2.25 + 0 + 0.25 + 1.00)/6 = √4.75/6 = √0.7917 ≈ 0.890*.

RMSE > MAE (*0.890 > 0.75*) because RMSE squares errors before averaging, magnifying the impact of the largest error (*1.5*). MAE treats all errors linearly.

A7. **

1. **Gathering ratings.** Solved by explicit asks (5-star rating UI) and implicit signals (purchase, watch-time inferred ratings).
2. **Extrapolating unknowns.** Solved by content-based filtering (item-feature cosine), collaborative filtering (Pearson similarity + neighbour weighted average), latent-factor models (matrix factorization).
3. **Evaluating.** Solved by holding out a random subset of known ratings, measuring RMSE / Precision@k / Spearman rank correlation between predictions and held-out values.

A8. Item-item beats user-user for three reasons. (1) More ratings per item than per user **— popular movies have hundreds of ratings while most users have rated only a handful, giving item similarities lower variance. (2)** Items have stable identity **— a movie is what it is, while users' tastes drift over time (mood, life stage). Item similarity is computed on a more stationary signal. (3)** Pre-computability **— the item set changes slowly, so item-item similarities can be cached and reused; user-user similarities shift every time anyone adds a rating, making caching impractical. (4 — bonus) Users have** multi-modal tastes** (love sci-fi AND romcoms) and similarity averaging dilutes that, while items have one identity that similarity captures cleanly.

A9. **

- **(a) New item.** Content-based wins easily: feature vector is known at item creation. CF cannot reason about an item with no ratings.
- **(b) New user.** Both struggle. Content-based has nothing to build a user profile from; CF has no neighbours to compare to. Both need fallback strategies (popularity, demographics, onboarding questions).
- **(c) Sparse matrix.** CF suffers most — finding users with overlapping rated items is hard. Content-based is unaffected because it doesn't need user-item overlap; each user is profiled independently via their own ratings.

A10. Use absolute values because Pearson similarities can be negative**. Example: two neighbours with sim = *+0.5* and sim = *-0.5*. With Σs_{ij} in the denominator: *0.5 + (-0.5) = 0*, prediction is undefined (division by zero). With $\Sigma |s_{ij}| = 0.5 + 0.5 = 1.0$, prediction is well-defined and correctly weighted by similarity magnitude. The point: a negative similarity contributes opposite information that we still want to use, but its magnitude matters in the normaliser.

A11. ** Cosine similarity is *fracx · y\|x\| \|y\|*. Pearson is *frac(x - ¯ x)·(y - ¯ y)\|x - ¯ x\| \|y - ¯ y\|*. Substitute *x' = x - ¯ x*, *y' = y - ¯ y* (mean-centred vectors). Then Pearson = *fracx' · y'\|x'\| \|y'\|* — exactly cosine similarity applied to the centred vectors. So Pearson IS centred cosine.

A12. (C)**. The formula sums only over users who actually rated *i* AND are in the top-*k* similar set. (A) is wrong: sum is restricted to top-*k*. (B) is wrong: must have rated *i*. (D) is wrong: don't include "0" for unrated items.

A13. (D)**. RMSE is *always greater than or equal to* MAE for the same error distribution (by Cauchy-Schwarz / Jensen's inequality applied to the convex function *x mapsto* x^2). So claim (D) — "RMSE always *le* MAE" — is false.

A14. True.** Content-based filtering computes similarity between the new item's *feature vector* and each user's profile vector, both of which exist independently of any rating data on the new item. Collaborative filtering relies on the rating column of the item; if no ratings exist, the similarity-with-other-items formulas have no data to operate on. This is the new-item cold-start problem.

A15. (B)**. The top-*k* neighbours are chosen by the** largest absolute** similarity, because negative correlations carry as much information as positive ones (just in the opposite direction). *|sim| = (0.6, 0.4, 0.3)* → B (*0.6*) and C (*0.4*) are top-2.

---

§13 — Ending Key Notes (Revision Cards)

Term	Quick-fact		
Utility matrix *U*	Sparse user × item rating table. Most cells empty — that's the whole challenge.		
Three pr oblems	(1) Gather ratings, (2) Extrapolate, (3) Evaluate.		
Explicit vs implicit ratings	Explicit = direct ask (rare). Implicit = inferred from behaviour (plentiful, noisy).		
Cold start	New items / new users have no ratings — collaborative methods break.		
Content -based recipe	Item profile (features) → user profile (avg of liked items) → cosine prediction.		
TF-IDF	$w_{ij} = TF_{ij} \cdot log(N/n_i)$. Pick top-weighted words as document profile.		
User-us er CF	Predict $\hat r(x, i) = \sum s_{xy} r_{yi} / \sum	s_{ xy }	*over top*-k$ neighbours.
Item-ite m CF	$\hat r(x, i) = \sum s_{ij} r_{xj} / \sum	s_{ ij}	*over* k *items rated by* x$.

Term	Quick-fact		
Item-item beats user-user	More ratings per item, items are stable, similarities pre-computable.		
Pearson similarity	Mean-centred cosine. Operates only on co-rated items.		
Cosine vs Pearson	Identical formulas — Pearson is just cosine on mean-centred vectors.		
Why $\sum	s	$ denominator	Negative similarities can sum to 0 → division by zero. Absolute values fix this.
Baseline predictor	$b_{xi} = \mu + b_x + b_i$. $b_x = \bar{r}_x - \mu$, $b_i = \bar{r}_i - \mu$.		
Joe / Sixth Sense	*3.7 + (-0.2) + 0.5 = 4* — pure baseline with no neighbours.		
Hybrid	Linear combination of content-based + CF + baseline. Netflix-prize-style.		
RMSE	$\sqrt{\frac{1}{N}\Sigma(\hat{r} - r^*)^2}$. Penalises outliers heavily.		
MAE vs RMSE	RMSE *ge* MAE always; equal only when all errors equal in magnitude.		
Mistake — sim denominator	Use $\sum	s_{ij}	*NOT* \sum s_{ij}$. Negative sims would cancel.
Mistake — neighbour set	Item-item: items rated by user *x*. User-user: users who rated item *i*.		
Mistake — missing = 0	Raw cosine treats blanks as 0 → biased low. Always centre first.		
LSH for speed	Use LSH (W4-5) to find top-*k* neighbours in sublinear time on huge user bases.		

§14 — Formula & Algorithm Reference

Concept	Formula	When to use				
Utility function	$u : X \times S \to R$	Defining the recommender problem space.				
TF-IDF	$w_{ij} = TF_{ij} \cdot log(N/n_i)$	Building text-item profiles.				
Cosine similarity (raw)	$\cos(x, y) = \frac{x \cdot y}{\	x\	\	y\	}$	Quick similarity on dense / non-rating vectors.
Pearson correlation	$frac\Sigma (r_{xs} - \bar r_x)(r_{ys} - \bar r_y)\sqrt\Sigma(r_{xs} - \bar r_x)^2\sqrt\Sigma(r_{ys} - \bar r_y)^2$	Standard CF similarity on rating data.				
User-user prediction	$\hat r(x, i) = \frac{\sum_{y \in N} s_{xy} r_{yi}}{\sum_{y \in N}	s_{xy}	}$	When more users than items, or in tutorial/exam questions.		
Item-item prediction	$\hat r(x, i) = \frac{\sum_{j \in N(i;x)} s_{ij} r_{xj}}{\sum_{j \in N(i;x)}	s_{ij}	}$	Default in production — items are simpler than users.		
Baseline predictor	$b_{xi} = \mu + b_x + b_i$	Quick guess when no neighbours; warm-start.				
Baseline + neighbours	$\hat r_{xi} = b_{xi} + \frac{\sum s_{ij}(r_{xj} - b_{xj})}{\sum	s_{ij}	}$	Best-of-class hand-coded predictor; Netflix prize.		
Hybrid linear combination	$\hat r_{xi} = w_1 \hat r^{(C)} + w_2 \hat r^{(CF)} + w_3 b_{xi}$	Production systems. Tune weights on validation set.				

Concept	Formula	When to use				
RMSE	*√frac1NΣ(^ r - r^*)^2*	Default eval metric. Penalises outliers heavily.				
MAE	$\frac{1}{N}\sum	\hat r - r^*	$	Robust alternative; equal weight to all errors.		
Precision @ k	*frac\#liked items in top-kk*	UX-driven evaluation when only top picks matter.				

Algorithmic complexity:

- User-user similarity matrix: $O(|X|^2 \cdot |S|)$ to compute, $O(|X|^2)$ to store.
- Item-item similarity matrix: $O(|S|^2 \cdot |X|)$ to compute, $O(|S|^2)$ to store. For most domains $|S| \ll |X|$ so item-item is cheaper.
- Per-prediction cost (with cached similarities): $O(k)$ — iterate over top-k neighbours.
- LSH-accelerated top-k search: sublinear in $|X|$ or $|S|$.

Connections to other weeks:

- **W4-5 — LSH:** Fast nearest-neighbour search for the top-k similar users / items, replacing the naive $O(N)$ scan.
- **W8-9 — PageRank:** Same eigenvector machinery underlies more sophisticated recommenders (e.g. ItemRank, random-walk-based CF).
- **MMDS Chapter 9:** Reference for matrix-factorization / latent-factor models (out of scope here but the next natural step beyond CF).

End of W07 Recommendation Systems exam-prep document.